



PROGRAMACIÓN WEB



```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Programación Web</title>
</head>
<body>
  <h1>Hola Mundo</h1>
</body>
</html>
```

HTML • CSS • JAVASCRIPT

HTML



CSS



JS



```
body {
  margin: 0;
  font-family:
    'Segoe UI',
    sans-serif;
  background:
    #f5f7fa;
}
```

```
// JavaScript
document.addEventListener
('DOMContentLoaded', () => {
  const btn = document.
    getElementById('btn');
  btn.addEventListener
    ('click', () => {
      alert('¡Hola Web!');
    });
});
```



INTEGRANTES

Iturralde Santiago

Genco Tomas

Pardo Juan



AÑO
2026



1) Definición del dominio

Tema elegido: Sistema de E-commerce (Catálogo, Usuarios y Carrito de Compras).

Para nuestra aplicación web, decidimos desarrollar un sistema base para un e-commerce. Esta aplicación funcionará como una plataforma que no solo permitirá administrar el catálogo de productos a través de un panel (manteniendo actualizado el inventario de la tienda), sino que también tendrá para gestionar usuarios y sus respectivos carritos de compras.

Para representar este dominio, definimos las siguientes entidades principales y la información que guardaremos para cada una:

Entidad: Producto Representa los artículos a la venta (por ejemplo, perfumes).

- **idProducto:** Identificador único para cada producto en la base de datos.
- **Nombre:** Título del artículo.
- **Descripción:** Detalle de las características principales.
- **Precio:** Valor de venta al público.
- **Stock:** Cantidad de unidades disponibles actualmente en el inventario.
- **Categoría:** Clasificación básica para ordenar los artículos.

Entidad: Carrito Representa la orden de compra en curso de un usuario.

- **idCarrito:** Identificador único del carrito.
- **idProductosCarrito:** Referencia a los ítems que contiene.
- **total:** Monto final calculado a pagar.

Entidad: ProductosCarrito Entidad intermedia que relaciona los productos con el carrito (detalle del carrito).

- **idProductosCarrito:** Identificador único del registro.
- **idProducto:** Referencia al producto específico que se está comprando.
- **idCarrito:** Referencia al carrito al que pertenece este ítem.
- **cantidad:** Número de unidades seleccionadas de ese producto.

Entidad: Usuario Representa a los clientes o administradores registrados en la plataforma.

- **idUsuario:** Identificador único en el sistema.
- **email:** Correo electrónico de contacto y validación.
- **contraseña:** Clave de seguridad para el ingreso a la cuenta.

2) Persistiendo el dominio

1. Diseño de la Base de Datos:

Entidad: producto

- id_producto: PRIMARY KEY, para que PostgreSQL autogenera un número entero único e incremental para cada perfume
- nombre_prod VARCHAR(255) NOT NULL, para almacenar el nombre del perfume
- descripcion_prod TEXT, porque las descripciones de perfumes requieren textos más largos y flexibles.
- precio_prod DECIMAL(10, 2) NOT NULL, evita errores de redondeo
- stock_prod INT NOT NULL DEFAULT 0, para contar las unidades físicas disponibles en el inventario, iniciando en cero por defecto
- categoria VARCHAR(100) NOT NULL, para clasificar los perfumes

Entidad: usuario

Representa a los clientes registrados o administradores del sistema e-commerce

- id_usuario (PRIMARY KEY): Para que PostgreSQL autogenera un número entero único e incremental, garantizando que cada cuenta de usuario registrada tenga un identificador irrepetible
- email (VARCHAR(255) UNIQUE NOT NULL): Almacena el correo electrónico que el usuario usará para iniciar sesión y recibir notificaciones. Se define como UNIQUE para evitar que existan dos cuentas registradas con el mismo correo, y NOT NULL porque es el dato obligatorio de autenticación.
- contraseña (VARCHAR(255) NOT NULL): Almacena la clave de acceso del usuario.

Entidad: carrito

Representa el carro activo en el cual un usuario va acumulando sus perfumes antes de pasar al pago

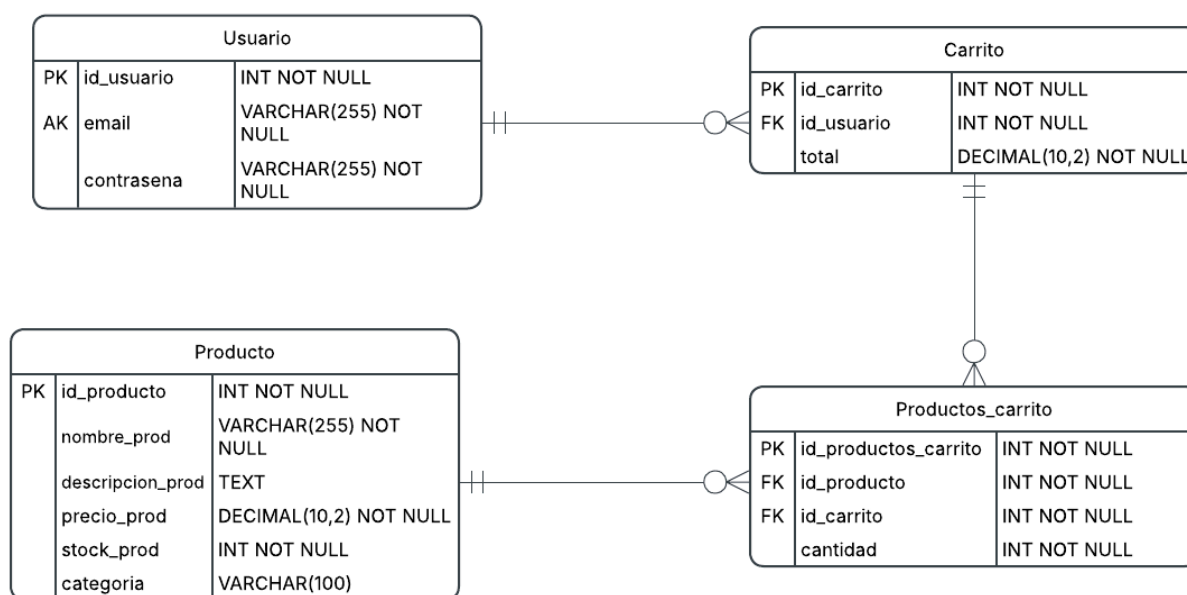
- id_carrito (PRIMARY KEY): Para que PostgreSQL autogenera un número entero único para identificar individualmente a cada carrito de compras del sistema
- id_usuario: Clave foránea (Foreign Key) que vincula de manera relacional el carrito de compras con el usuario específico que lo está armando.
- total (DECIMAL(10, 2) NOT NULL DEFAULT 0.00): Almacena el monto final calculado de la compra. Al igual que con el precio de los perfumes, se utiliza DECIMAL(10, 2) para asegurar precisión y evitar errores

Entidad: productos_carrito

Es la tabla intermedia para resolver la relación entre perfumes y carritos

- id_productos_carrito (PRIMARY KEY): Identificador autogenerado por PostgreSQL para cada línea o ítem individual que se agrega a un carrito

- **id_producto**: Clave foránea que asocia de forma obligatoria este ítem con el perfume específico seleccionado. Si el producto llegara a ser eliminado del catálogo general de perfumes, la base de datos automáticamente eliminará ese artículo de todos los carritos activos de los clientes.
- **id_carrito**: Clave foránea que indica a qué carrito de compras pertenece esta línea de detalle. Si el carrito se elimina (porque el cliente canceló la compra o vació su carro), todas sus líneas de productos asociadas se borrarán en cascada.
- **cantidad** (INT NOT NULL DEFAULT 1): Registra el número de unidades físicas de ese perfume en particular que el cliente ha decidido



llevar.

2. Definición de Consultas:

Para que nuestra e-commerce se conecte con la base de datos, decidimos usar sqlc, ya que lee nuestras consultas de SQL y nos genera el código de Go de manera automática. En esta parte nos encargamos de definir el comportamiento para las tablas de producto, usuario, productos_carrito y carrito.

¿Cómo lo hicimos?

- Escribimos las consultas para las acciones básicas (crear, buscar, listar, actualizar y borrar). Para que la herramienta entienda qué tipo de código Go debe generar, le agregamos comentarios especiales arriba de cada consulta
 1. `:one` para cuando esperamos un solo resultado (como buscar un perfume por su ID o crear un usuario)
 2. `:many` para cuando queremos traer una lista (como listar todo el catálogo de perfumes)
 3. `:exec` para acciones que solo modifican datos sin devolver registros (como actualizar un stock o borrar un usuario)

Además, usamos marcadores de posición (\$1, \$2) para los datos que va a ingresar el usuario, lo hicimos de esta manera por:

- Seguridad: Al usar parámetros como \$1 o \$2 separamos las instrucciones SQL de los datos de entrada, bloqueando por completo los ataques de inyección SQL
- Ahorro de tiempo: Nos evita escribir un montón de código repetitivo para mapear los datos de las tablas a variables de Go. Si nos equivocamos en el nombre de una columna, el compilador nos avisa al instante

3. Creación schema.sql

Se creó el archivo db/schema/schema.sql para la creación de las tablas en la DB PostgreSQL.

4. Creación queries.sql

Se creó el archivo db/queries/queries.sql que contiene operaciones CRUD para todas las entidades de la DB, para el posterior uso del compilador sqlc. Este compilador nos va a ayudar a realizar el mapeo ORM de manera automática sin intervención del equipo de desarrollo, solo viendo las consultas escritas en dicho archivo.

5. Generación de código de Acceso de datos

se creó el archivo sqlc.yaml indicando el motor de base de datos, el esquema a utilizar y el archivo donde se encuentran las queries, luego se aclaró como se generara el código GO, indicando el paquete, donde se guardaría el código y se4 habilitó la opción de agregar etiquetas JSON a los structs generados por sqlc.

Se configuró la herramienta sqlc para que lea el esquema de la base de datos y utilice las consultas previamente hechas, utilizando las anotaciones que requiere sqlc (-- name), así luego generando el código GO para ejecutarlas.

se ejecuto el comando sqlc generate, que utilizó la configuración definida anteriormente para generar automaticamente el código Go correspondiente al esquema y a las consultas SQL, se crearon 3 archivos Go dentro de la ruta /db/sqlc

-[db.go](#): contiene la estructura que usa el código generado para poder ejecutar las consultas para utilizar la base de datos

-[models.go](#): define la estructura de cómo se representan los datos (Producto, Carrito, ProductosCarrito y Usuario)

-[queries.sql.go](#): contiene las funciones Go que sqlc género a partir de las consultas del archivo db/queries.sql

Implementación de la capa de persistencia

Se utiliza PostgreSQL para almacenar de forma permanente los datos de usuarios, productos y carritos. Para acceder a estos datos desde la aplicación se utiliza el código generado anteriormente mediante sqlc, que permite ejecutar las consultas definidas en SQL desde Go.

se incorporó la capa Repository para centralizar el acceso a la base de datos, en lugar de ejecutar las consultas directamente desde los handlers, el repository utiliza funciones generadas por sqlc para comunicarse con PostgreSQL

se creó la capa service, esta se encuentra entre los handlers y los repositorios, se encarga de aplicar la lógica de negocio, válida o normalizar los datos y coordinar las operaciones necesarias antes de acceder a la persistencia, en el caso de una alta de usuarios, UsuariosService procesa los datos recibidos por UsuarioHandler y utiliza UsuarioRepository para almacenarlos

PostgreSQL mediante Docker Compose

Para ejecutar la base de datos se utilizó Docker Compose, levantando PostgreSQL dentro de un contenedor local, se configuró la imagen postgres:14-alpine y se definieron las credenciales de conexión mediante variables de entorno almacenadas en el archivo .env, evitando dejarlas escritas directamente en el código, el Contenedor expone el puerto 5432, permitiendo que la aplicación Go se conecte a PostgreSQL desde localhost.

Se configuró un volumen de docker para almacenar los datos de PostgreSQL de forma persistente. esto permite que la información de la base de datos no se pierda aunque el contenedor se detenga o se elimine mediante un “docker compose down”, cuando el contenedor vuelve a levantarse, PostgreSQL reutiliza el mismo volumen y recupera los datos almacenados

Automatización mediante MakeFile

Para simplificar la ejecución de las distintas tareas se creó un archivo MakeFile. En este archivo se definieron los distintos targets que permiten agrupar comandos y ejecutarlos de manera más sencilla mediante make.

entre los principales se encuentran “generate”, que ejecuta sqlc generate, “build” que genera nuevamente código necesario y compila la aplicación, “run” utilizado para iniciar Air, “test” encargado de preparar el entorno de pruebas y ejecutar los test y “clean” utilizado para eliminar archivos temporales generados durante la compilación

Uso de air para el desarrollo

se incorporó la herramienta Air, esta se encarga de observar los cambios realizados en el proyecto y recompila automáticamente la aplicación cuando detecta una modificación, de esta manera no es necesario detener y volver a ejecutar manualmente el servidor cada vez que hay algún cambio

la configuración se realizó mediante el archivo `.air.toml`, donde se indican las extensiones de archivos a observar, las carpetas a ignorar y el comando que se debe ejecutar ante un cambio, en nuestro caso Air utiliza `make build`

Pruebas de persistencia

para comprobar si la persistencia funciona correctamente se agregaron test utilizando el paquete testing de Go, estas pruebas utilizan el flujo real de la aplicación y permiten comprobar que las operaciones sobre la base de datos se comporten como se espera

Entre los casos verificados se encuentra el alta correcta de un usuario, el rechazo de datos inválidos sin generar registros en la base y la validación de los emails duplicados, también se comprueba que la respuesta de la aplicación no exponga información sensible como la contraseña de usuario.

la ejecución de estas pruebas se integro al target `"make test"`, que prepara automáticamente el entorno necesario, levanta PostgreSQL mediante Docker, carga el esquema, ejecuta los test y finalmente elimina los contenedores y volúmenes utilizados, de esta forma las pruebas pueden repetirse siempre sobre un entorno limpio y sin afectar la base de datos utilizada durante el desarrollo